

AUDITORÍA TÉCNICA PROFUNDA

GSentinelHealthOS + medical-agenda-saas

Fecha: 23 de Abril de 2026 | Versión: 1.0 | Clasificación: CONFIDENCIAL

Ingeniero Auditor: GitHub Copilot — Especialista HealthTech & IA Médica

DIMENSIÓN	NIVEL ACTUAL	ESTADO
Motor de IA	Nivel 3 — Decisión Contextual	■ PARCIAL
Independencia MetaBrain	AUTÓNOMO (local)	✓ OK
Agenda Médica	Nivel PRO — Optimización real	✓ SÓLIDO
Dashboard Secretaria	Nivel Medio-Alto	■ MEJORAS
Dashboard Médico	Funcional, mejoras pendientes	■ MEJORAS
Pipeline WhatsApp	DLQ + Retry implementados	✓ OK
Multi-tenant SaaS	Implementado en medical-agenda	✓ OK
Microservicios IA externos	NO DESPLEGADOS	✗ CRÍTICO
Veredicto Sistema	PRE-PRODUCCIÓN AVANZADO	■ NO PROD

■ 1. ARQUITECTURA GENERAL

Patrón detectado

Arquitectura **híbrida event-driven + monolito modular**. El sistema combina un worker Python (Brain) consumiendo colas Redis con un frontend Next.js 14 (medical-agenda-saas) que expone API REST propia. No hay un bus de eventos centralizado (ej. Kafka/RabbitMQ); Redis actúa como broker simplificado con listas (BRPOP/LPUSH).

Flujo completo mapeado

ETAPA	COMPONENTE	TECNOLOGÍA	ESTADO
1. Entrada	WhatsApp Cloud API webhook	HTTPS POST → gateway	✓ OK
2. Gateway	whatsapp_gateway / FastAPI	Python + Redis LPUSH	✓ OK
3. Cola entrada	whatsapp:incoming (Redis)	BRPOP blocking	✓ OK
4. Worker Brain	BrainWorker → BrainOrchestrator	Python asyncio	✓ OK
5. NLU / MetaBrain	NLUEngine (rule+fuzzy+KB)	Local — sin LLM	✓ OK
6. Cola salida	whatsapp:outgoing (Redis)	LPUSH + DLQ :dead	✓ OK
7. Envío WA	WhatsAppOutgoingConsumer	Graph API HTTP	✓ OK
8. API SaaS	medical-agenda-saas / Next.js	Prisma + PostgreSQL	✓ OK
9. Motor IA orq.	IntelligentOrchestrator	4 microservicios HTTP	✗ NO EXISTE

Acoplamientos y SPOF detectados

- **Redis (single node):** CRÍTICO — Es el único broker. Sin Redis, el sistema de mensajería WhatsApp colapsa completamente. No hay failover ni Redis Sentinel configurado en docker-compose.yml.
- **PostgreSQL (single node):** ALTO — Un solo contenedor DB. Sin replicación. Pérdida de datos ante fallo de volumen.
- **WhatsApp Graph API:** MEDIO — Dependencia total de Meta. Sin circuit breaker a nivel gateway (el CB está en el orchestrator de IA, no en el envío real).
- **IntelligentOrchestrator:** CRÍTICO — Referencia a 4 microservicios (dialogue-engine, inference-service, decision-service, nlq-service) que NO están implementados como servicios reales en el repo. Son clientes HTTP a endpoints inexistentes.

■ 2. ANÁLISIS DE IA — ESTADO REAL

2.1 — Dos orquestadores en conflicto (HALLAZGO CRÍTICO)

El repositorio contiene **DOS orquestadores distintos** que coexisten sin coordinación explícita:

ORQUESTADOR	ARCHIVO	TIPO	ESTADO REAL
BrainOrchestrator	brain/services/orchestrator.py	NLU rule-based + API calls	ACTIVO en producción
IntelligentOrchestrator	brain/orchestration/orchestrator.py	4 microservicios HTTP externos	INACTIVO — servicios no existen

2.2 — MetaBrain: ¿servicio externo o motor interno?

VEREDICTO: MetaBrain es 100% interno y local. No es un servicio externo, no hay llamadas HTTP salientes, no hay SDK de terceros. 'MetaBrain' es el nombre interno del motor NLU definido en *brain/interpreters/nlu_engine.py*. Funciona completamente offline con:

- ✓ Clasificación de intención mediante diccionarios de palabras clave (rule-based)
- ✓ Fuzzy matching con SequenceMatcher de Python stdlib (umbral 0.65–0.95)
- ✓ LessonCache con TTL de 5 minutos para lecciones del doctor
- ✓ KnowledgeMatcher para búsqueda exacta y aproximada en Knowledge Base
- ✓ Circuit breakers por servicio (shared.utils.resilience — código propio)

2.3 — Clasificación del nivel de IA

CAPA	COMPONENTE	NIVEL	DETALLE
NLU / Intención	NLUEngine (MetaBrain)	Nivel 2–3	Reglas + fuzzy + context boost
Generación NLG	LinguisticEngine	Nivel 2	Templates con variabilidad aleatoria, sin LLM
Decisión conversac.	BrainOrchestrator	Nivel 2	FSM de estados (booking, cancel, etc.)
Orquestación avanzada	IntelligentOrchestrator	Nivel 3 (código)	Código implementado pero microservicios destino AUSENTES
Predicción no-show	predictionEngine.ts (ONNX)	Nivel 4	Modelo ML real, sigmoid, features históricas
Autoasignación turnos	appointmentEngine.ts	Nivel 3–4	Scoring multicriterio: fragmentación, preferencia, no-show prob
Imaging médica	predictor.service.ts (ONNX)	Nivel 3–4	ONNX con fallback estructurado, softmax, multilabel

Hallazgo crítico: decision_engine/ está VACÍO

La carpeta **brain/decision_engine/** no contiene ningún archivo. El motor de decisión clínica prometido en la arquitectura (triage, risk_level, flags) se genera únicamente como *fallback estático* en el orquestador (`_FALLBACK_DECISION = {risk_level: 'unknown', ...}`). No hay lógica real de triage médico implementada.

■ 3. PIPELINE WHATSAPP

Estado del pipeline de mensajería

COMPONENTE	ESTADO	EVIDENCIA
Webhook entrada	✓ Implementado	whatsapp_gateway / FastAPI + verify token
Cola Redis entrada	✓ Implementado	whatsapp:incoming — BRPOP blocking
Retry en salida	✓ Implementado	max_retries=5, requeue RPUSH
Dead Letter Queue	✓ Implementado	whatsapp:outgoing:dead — LPUSH tras 5 fallos
Circuit Breaker (Brain)	✓ Implementado	CircuitBreakerRegistry por servicio
Circuit Breaker (Gateway)	✗ AUSENTE	El outgoing_consumer no tiene CB propio
Alertas DLQ en tiempo real	✗ AUSENTE	No hay webhook/notificación cuando crece DLQ
Idempotencia de mensajes	■ PARCIAL	No hay deduplicación por message_id de Meta
Rate limiting WhatsApp	■ PARCIAL	RateLimit en DB pero no se verifica antes de enviar
Logging estructurado	✓ Implementado	mask_phone() presente, JSON logs

Riesgo principal: Si la DLQ crece silenciosamente (sin alertas), los mensajes fallidos de pacientes se pierden sin notificación. En un entorno médico, un paciente que no recibe confirmación de turno puede presentarse sin cita o ausentarse sin cancelar — impacto operativo directo.

■ 4. AGENDA MÉDICA — CORE DEL NEGOCIO

Nivel del motor de agenda: PRO con matices

El **appointmentEngine.ts** es el componente más maduro del sistema. Implementa un motor de autoasignación con scoring multicriteria real, no CRUD simple.

FUNCIONALIDAD	ESTADO	NIVEL
Slots dinámicos por regla de disponibilidad	✓ Implementado	PRO
Control de solapamiento con DB lock	✓ Implementado (lockDoctorForScheduling)	PRO
Predicción de no-show con ONNX	✓ Implementado	PRO
Overbooking inteligente condicional	✓ Implementado (env configurable)	PRO
Score de fragmentación de agenda	✓ Implementado (fragmentationScore)	PRO
Preferencia horaria (mañana/tarde)	✓ Implementado	MEDIO
Autoasignación de médico disponible	✓ Implementado (findDoctorCandidates)	PRO
Reglas por día semana + fecha específica	✓ Implementado	PRO
Buffer entre turnos configurable	■ Implícito en slot_duration	MEDIO
Reglas por tipo de consulta (urgencia/control)	✗ AUSENTE	FALTA PRO
Google Calendar sync bidireccional	■ Mencionado en docs, no en engine	PENDIENTE
Cancelación automática por no confirmación	✗ AUSENTE	FALTA PRO
Recordatorios automáticos pre-turno (WA)	✗ AUSENTE en agenda engine	FALTA PRO

■ 5. DASHBOARDS — SECRETARIA Y MÉDICO

5.1 — Dashboard Secretaria

Implementado en **SecretariaDashboard (secretaria-dashboard.tsx)**. Es el dashboard más completo del sistema.

FUNCIONALIDAD	PRESENTE	INTELIGENCIA
FullCalendar con drag & drop de turnos	✓ SÍ	UI funcional
Vista semanal y mensual	✓ SÍ	UI funcional
Crear/editar turnos inline	✓ SÍ	Formulario completo
Gestión de reglas de disponibilidad	✓ SÍ	Con slot_duration por regla
Panel mensajes fallidos WhatsApp	✓ SÍ (FailedMessagesPanel)	Vista de errores de mensajería
Importar agenda desde imagen/texto (AI Intake)	✓ SÍ (OCR+parsing)	IA real — interpreta PDFs/fotos
Alertas de pacientes en espera	■ Solo contador en Dashboard	Sin alertas push
Sugerencias inteligentes de horario	✓ SÍ (via /api/recommendations)	Scoring real con no-show
Auto-asignación de turno	✓ SÍ (/api/appointments/autoassign)	Machine scoring completo
Notificación en tiempo real (WebSocket)	✗ AUSENTE	Solo polling cada 30s
Vista de ocupación por médico	■ Solo en Dashboard overview	Básica
Filtro por especialidad	✓ SÍ	Funcional

5.2 — Dashboard Médico

Implementado en **DoctorDashboard (doctor-dashboard.tsx)**. Orientado a la jornada diaria del médico.

FUNCIONALIDAD	PRESENTE	EVALUACIÓN
Turnos del día (auto-refresh 30s)	✓ SÍ	Actualización automática
Historial de visitas previas del paciente	✓ SÍ	Desde BD, ordenado por fecha
Notas clínicas del paciente (editable)	✓ SÍ	Campo libre + guardado
Registro de evolución clínica	✓ SÍ	Se concatena a notes del turno
Reprogramar turno desde el panel	✓ SÍ	Con fecha/hora + duración
Crear turno de seguimiento	✓ SÍ (/api/appointments/createfollowup)	Con followup intervalo
Chat clínico con MetaBrain (IA)	✓ SÍ	Historial persistido, respuestas contextuales
Chat libre SIN turno seleccionado	✗ AUSENTE	Requiere selectedAppointment activo
Análisis de imagen médica (ONNX)	✓ SÍ (en panel imaging)	ONNX + fallback estructurado
Alertas por riesgo clínico	✗ AUSENTE	No hay badges de riesgo en lista de turnos
Soporte a decisión (sugerencias IA)	■ Solo chat libre	Sin panel de sugerencias estructurado
Vista de estadísticas propias del médico	✗ AUSENTE	No hay métricas individuales

Nota sobre chat médico: La memoria del usuario indica que el chat del doctor debe ser LIBRE (sin requerir turno seleccionado). El código actual en *doctor-dashboard.tsx* fuerza `selectedAppointment` como requisito para cargar el historial de chat. Esto bloquea el uso libre del chat entre consultas.

■ 6. REDIS / WORKERS / CONCURRENCIA

Estado de Redis y manejo de colas

- Cola entrada (*whatsapp:incoming*): OK — BRPOP con timeout=5s
- Cola salida (*whatsapp:outgoing*): OK — retry + requeue
- DLQ salida (*whatsapp:outgoing:dead*): OK — LPUSH tras 5 reintentos
- Estado conversacional (*chat_state:{phone}*): OK — TTL configurable (settings.state_ttl_seconds)
- Bot pausado (kill-switch) (*bot_paused:{phone}*): OK — toggle manual
- Métricas del Brain (*brain:metrics:{name}*): OK — contadores con incrby
- Cache sesiones IA (*session:{id}*): Implementado en OrchestratorSessionManager
- Memoria semántica (*semantic_memory:{session}*): Implementado en SemanticMemory

Riesgos detectados en Redis

✗ **[CRÍTICO] Sin Redis Sentinel/Cluster:** Single node Redis. Si cae, el sistema de mensajería cae completo. No hay configuración de HA en docker-compose.yml.

■ **[ALTO] Sin persistencia garantizada:** Redis por defecto (docker redis:7-alpine) no tiene AOF habilitado. Mensajes en cola pueden perderse ante restart.

■ **[BAJO-MEDIO] Race condition en StateManager:** El bloqueo usa Lua script para unlock pero la adquisición del lock tiene retry sin backoff exponencial (sleep fijo).

■ **[MEDIO] Bulkhead inference: semaphore=5:** Si inference-service existiera, máximo 5 requests concurrentes. Con múltiples workers escalaría como cuello de botella.

■ **[MEDIO] Sin monitoreo de longitud de cola:** No hay alerta si whatsapp:incoming crece. Un burst de mensajes puede acumular sin notificación.

■ 7. MOTOR NLG — GENERACIÓN DE LENGUAJE

El **LinguisticEngine** (brain/orchestration/linguistic_engine.py) es un generador de lenguaje propio sin LLM.

Evaluación técnica:

CRITERIO	ESTADO	DETALLE
Variabilidad de respuestas	✓ Implementado	_pick_unique() evita repetir la misma frase
Contexto clínico en respuesta	✓ Parcial	Usa risk_level, triage, symptoms si los hay
Templates dinámicos	✓ SÍ	risk_templates, triage_templates, symptom_templates
Frases de emergencia	✓ SÍ	emergency_phrases aleatorias
Adaptación casual vs clínico	✓ SÍ	detect_input_type() → shortcut casual o pipeline clínico
Generación sin contexto suficiente	✓ SÍ	low_information_phrases con variabilidad
Español rioplatense natural	✓ SÍ	Tuteo, 'contas', 'vos', sin robotismo
Soporte multilenguaje	✗ AUSENTE	Solo español. Sin configuración de idioma.
Personalización por paciente	✗ AUSENTE	No usa nombre del paciente en respuestas WA
Conector con historial de conversación	■ Parcial	History se pasa pero NLG no lo lee directamente
Respuestas estructuradas (listas, botones)	✗ AUSENTE	Solo texto plano. Sin interactive messages WA

Evaluación: El NLG es adecuado para un MVP en producción. La mayor limitación es la ausencia de WhatsApp Interactive Messages (botones, listas) que mejorarían significativamente la tasa de conversión en el flujo de booking.

■ 8. PROBLEMAS CRÍTICOS — TOP 5

#1 BLOQUEANTE — Microservicios IA inexistentes

El IntelligentOrchestrator (brain/orchestration/orchestrator.py) referencia 4 servicios HTTP (dialogue-engine, inference-service, decision-service, nlg-service) mediante clientes HTTP con circuit breakers y retry. Estos servicios NO EXISTEN como contenedores ni código en el repo. El sistema funciona porque el BrainOrchestrator (services/orchestrator.py) es el que realmente se usa en el worker, pero si alguien activa el IntelligentOrchestrator todos los requests fallan y caen al fallback estático. Nivel de riesgo: ARQUITECTURAL.

#2 CRÍTICO — Redis sin alta disponibilidad

El broker de mensajería completo (incoming/outgoing queues, state, metrics) depende de un único nodo Redis sin Sentinel, sin Cluster, sin AOF persistencia. Un restart del contenedor Redis durante un pico de mensajes implica pérdida irreversible de conversaciones activas y mensajes pendientes. En un sistema médico esto equivale a citas perdidas.

#3 CRÍTICO — Chat médico no funciona sin turno seleccionado

El DoctorDashboard requiere que haya un selectedAppointment activo para cargar el chat con MetaBrain (loadChatHistory usa selectedAppointment.patient.id como prerequisite). Si el médico no tiene turnos ese día, o quiere hacer una consulta libre, el chat está bloqueado. La preferencia del usuario es explícita: el chat debe ser libre.

#4 ALTO — decision_engine/ vacío + triage siempre 'unknown'

La carpeta brain/decision_engine/ no tiene código. El campo triage_level retorna 'unknown' en todos los flujos de fallback. El sistema no tiene capacidad real de triaje clínico. Esto es un gap funcional importante para cualquier clínica que necesite priorización de urgencias.

#5 ALTO — Sin WebSockets / push notifications en paneles

Todos los dashboards usan polling HTTP (30s para el panel médico, 15s para el dashboard interno de GSentinelHealthOS). En un entorno clínico real con 10+ médicos y 100+ turnos simultáneos, el polling genera carga innecesaria y latencia en alertas críticas. Un turno agregado por WhatsApp tarda hasta 30s en aparecer en el panel de la secretaria.

■ 9. QUÉ FALTA PARA SER PRO

INFRAESTRUCTURA

- Redis Sentinel o Redis Cluster + AOF persistencia habilitada
- PostgreSQL con réplica de lectura + backup automático (pg_dump en cron)
- Health checks reales en docker-compose (no solo el CMD)
- Rate limiter Redis en gateway (no solo en DB) para proteger ante floods

IA y DECISIONES

- Implementar realmente los 4 microservicios del IntelligentOrchestrator O eliminar la capa y unificar en BrainOrchestrator
- Poblar brain/decision_engine/ con lógica de triage (Manchester Triage System simplificado: 5 niveles por síntomas clave)
- Conectar predictionEngine ONNX al BrainOrchestrator (hoy solo existe en medical-agenda-saas, no en el worker de WA)
- Activar WhatsApp Interactive Messages (botones de selección de horario, confirmación de turno)

PANELES Y UX MÉDICA

- Chat médico libre sin prerequisite de turno seleccionado (ya documentado como preferencia del usuario)
- Botón de borrar historial de chat desde la interfaz (preferencia del usuario)
- WebSockets en secretaria-dashboard para actualización en tiempo real de nuevos turnos WA
- Panel de análisis de imagen: informe médico preliminar (hallazgos/impresión/recomendaciones) en vez de solo metadatos
- Badges de riesgo clínico en lista de turnos del médico (bajo/medio/alto basado en historial)
- Estadísticas propias del médico: tasa de no-show, ocupación promedio, tiempo promedio de consulta

AGENDA MÉDICA

- Reglas de duración por TIPO de consulta (primera vez: 45min, control: 20min, urgencia: 15min)
- Sistema de recordatorios automáticos 24h antes via WhatsApp (job scheduler)
- Cancelación automática con ventana de gracia si el paciente no confirma
- Integración completa Google Calendar (sincronización bidireccional — documentado pero no implementado en engine)

SEGURIDAD Y COMPLIANCE

- Auditoría completa implementada (AuditLog existe en schema, verificar que TODOS los endpoints la usen)
 - Encriptación de datos sensibles en reposo (notas clínicas, documentos de pacientes)
 - HIPAA/GDPR compliance review: retención de datos, derecho al olvido, anonimización
 - 2FA para roles médico y admin
 - Firma digital de órdenes y evoluciones clínicas
-

■ 10. VEREDICTO FINAL

DIMENSIÓN	VEREDICTO	PUNTUACIÓN
Arquitectura general	Event-driven híbrida, SPOF en Redis	6.5 / 10
Motor de IA (autonomía)	Autónomo local. MetaBrain = interno. Nivel 3 real.	7 / 10
Independencia de MetaBrain	100% independiente. Sin servicios externos.	10 / 10
Motor IntelligentOrchestrator	Código completo PERO microservicios destino no existen	2 / 10
Agenda médica	Nivel PRO real. Scoring, ONNX, overbooking, locks.	8.5 / 10
Dashboard Secretaria	Funcional y con IA embebida. Falta push real-time.	7.5 / 10
Dashboard Médico	Útil clínicamente. Chat bloqueado sin turno.	6 / 10
Pipeline WhatsApp	DLQ + Retry correctos. Sin CB en gateway ni dedup.	7 / 10
NLG propio	Variabilidad buena. Sin botones interactivos.	6.5 / 10
Multi-tenant SaaS	Implementado correctamente en medical-agenda-saas.	8 / 10
Seguridad	Auth JWT, roles, audit log. Sin 2FA ni encriptación en reposo.	6 / 10
Preparación producción	Pre-producción avanzado. 3 blockers críticos.	5.5 / 10

DIAGNÓSTICO FINAL DEL SISTEMA

GSentinelHealthOS es un sistema de agenda médica con IA real, en estado **PRE-PRODUCCIÓN AVANZADO**. Tiene componentes genuinamente sólidos (motor de autoasignación con ONNX, multi-tenant, DLQ, circuit breakers, ONNX imaging) y componentes rotos o incompletos (IntelligentOrchestrator sin microservicios, decision_engine vacío, Redis sin HA, chat médico con requisito bloqueante).

¿Es SaaS real? SÍ — multi-tenant con Prisma, planes, límites y slug funcionan.

¿Es IA médica real? PARCIALMENTE — La predicción ONNX y el engine de autoasignación son IA real. El NLU del chatbot es rule-based avanzado. El triage clínico está AUSENTE.

¿Está listo para PRO? NO — Requiere resolver los 5 problemas críticos antes de ir a producción real.